

PROGRAMMATION EN FORTRAN

Licence 3 - Physique

Résumé du cours

1. Structure d'un programme Fortran.

```

1 program nom_programme
2   implicit none
3   ! declarations
4   ! instructions
5 end program nom_programme

```

La directive `implicit none` est indispensable : elle oblige la déclaration explicite de toutes les variables.

2. Types de base. `integer` (entier), `real` (flottant simple précision), `double precision` ou `real(8)` (double précision), `complex` (complexe), `logical` (booléen), `character` (chaîne).

3. Structures de contrôle.

- . Conditionnelle : `if (cond) then ... else if ... else ... end if`
- . Boucle `do` : `do i=debut, fin, pas ... end do`
- . Boucle `do while` : `do while (cond) ... end do`
- . Sortie de boucle : `exit` (sortie) et `cycle` (passage à l'itération suivante)

4. Tableaux. `real, dimension(n) :: tab` ou `real :: tab(n)`. Indices de 1 à n par défaut. Allocation dynamique : `allocatable`, `allocate`, `deallocate`.

5. Fonctions et sous-programmes.

- . `function nom(args) result(res)` : retourne une valeur.
- . `subroutine nom(args)` : ne retourne pas de valeur, modifie les arguments.
- . Appel d'une sous-routine : `call nom(args)`.

6. Entrées/Sorties. `read(*,*) x` (entrée standard), `write(*,*) x` ou `print*, x` (affichage).

Pour un fichier : `open(10, file='data.txt'), read(10,*) x, close(10)`.

1. Exercices fondamentaux

Exercice 1 - Résolution d'un système linéaire 2CE2

★

Exercice

Écrire un programme Fortran qui résout le système :

$$\begin{cases} u_1x + v_1y = w_1 \\ u_2x + v_2y = w_2 \end{cases}$$

par la méthode de Cramer. Le programme doit vérifier l'existence d'une solution unique et afficher le résultat.

```

1 program systeme
2   implicit none
3   real :: u1, u2, v1, v2, w1, w2
4   real :: delta, delta_x, delta_y, x, y
5
6   ! Valeurs des coefficients
7   u1 = 2.0; u2 = 4.0
8   v1 = 5.0; v2 = 11.0
9   w1 = 7.0; w2 = 6.0
10
11  ! Calcul du determinant principal
12  delta = u1*v2 - u2*v1
13
14  if (abs(delta) < 1.0e-6) then
15    print *, "Le systeme n'a pas de solution unique."
16    stop
17  end if
18
19  ! Determinants de Cramer
20  delta_x = w1*v2 - w2*v1
21  delta_y = u1*w2 - u2*w1
22
23  x = delta_x / delta
24  y = delta_y / delta
25
26  print *, "Solution : x =", x, ", y =", y
27 end program systeme

```

Résultat pour les valeurs données : $\delta = 22 - 20 = 2$, $\delta_x = 77 - 30 = 47...$
 On trouve $x = 47/2 = 23.5$? Vérifions : $u1 = 2$, $v1 = 5$, $u2 = 4$, $v2 = 11$, donc
 $\delta = 2 \times 11 - 4 \times 5 = 22 - 20 = 2$. $\delta_x = 7 \times 11 - 6 \times 5 = 77 - 30 = 47$, donc $x = 23.5$.
 $\delta_y = 2 \times 6 - 4 \times 7 = 12 - 28 = -16$, donc $y = -8$.

La règle de Cramer : pour $Ax = b$, $x_i = \det(A_i)/\det(A)$ où A_i est la matrice A dont la i -ème colonne est remplacée par b .

Exercice 2 - Racines d'un trinôme du second degré

★

Exercice

Écrire un programme Fortran qui calcule les racines (réelles) de $ax^2 + bx + c = 0$. Le programme doit :

1. Vérifier que $a \neq 0$.
2. Calculer le discriminant $\Delta = b^2 - 4ac$.
3. Afficher les racines si elles existent, ou un message approprié.

```

1 program trinome
2   implicit none
3   real, parameter :: epsilon = 1.0e-6
4   real :: a, b, c, delta, x1, x2
5
6   ! Valeurs des coefficients
7   a = 1.0; b = -5.0; c = 6.0 ! x^2 - 5x + 6 = 0
8
9   if (abs(a) < epsilon) stop "a doit etre non nul."
10
11  delta = b*b - 4.0*a*c
12
13  if (delta < -epsilon) then
14    print *, "Pas de racine reelle (discriminant negatif)."
15  else if (abs(delta) <= epsilon) then
16    x1 = -b / (2.0*a)
17    print *, "Racine double : x =", x1
18  else
19    x1 = (-b - sqrt(delta)) / (2.0*a)
20    x2 = (-b + sqrt(delta)) / (2.0*a)
21    print *, "Deux racines : x1 =", x1, ", x2 =", x2
22  end if
23 end program trinome
    
```

Pour $x^2 - 5x + 6 = 0$: $\Delta = 25 - 24 = 1 > 0$, racines $x_1 = 2$ et $x_2 = 3$.

Exercice 3 - Suite de Fibonacci et nombre d'or

★★

Exercice

La suite de Fibonacci est définie par $u_0 = u_1 = 1$ et $u_{n+1} = u_n + u_{n-1}$. Le rapport u_{n+1}/u_n converge vers le nombre d'or $\varphi = (1 + \sqrt{5})/2$.

Écrire un programme qui calcule les termes de la suite jusqu'à ce que $|u_{n+1}/u_n - u_n/u_{n-1}| < 10^{-5}$, puis affiche la valeur approchée et la valeur exacte du nombre d'or.

```

1 program nombre_dor
2   implicit none
3   real, parameter :: epsilon = 1.0e-5
4   real :: u_prec, u_cour, somme
5   real :: v_prec, v_cour, nombre_or
6   integer :: n
7
8   nombre_or = (1.0 + sqrt(5.0)) / 2.0
9
10  u_prec = 1.0; u_cour = 1.0
11  v_prec = 1.0; n = 1
12
13  do
14    somme = u_cour + u_prec
15    v_cour = (u_cour + u_prec) / u_cour
16    u_prec = u_cour
17    u_cour = somme
18    n = n + 1
19    if (abs(v_cour - v_prec) < epsilon) exit
20    v_prec = v_cour
21  end do
22
23  write(*, '(A,F12.8)') "Approximation (n = ", real(n), " ) :",
24    v_cour
25  write(*, '(A,F12.8)') "Valeur exacte",
26    nombre_or
27 end program nombre_dor

```

La convergence est atteinte après une vingtaine d'itérations. Le nombre d'or $\varphi \approx 1,61803399$.

Exercice 4 - Crible d'Ératosthène

★★

Exercice

Le crible d'Ératosthène détermine tous les nombres premiers jusqu'à n :

1. Créer un tableau de tous les entiers de 2 à n .
2. Pour chaque entier i de 2 à $\sqrt{n}$: si i n'a pas été rayé, déclarer i premier et rayer tous ses multiples.
3. Les entiers non rayés sont premiers.

Écrire ce programme en Fortran pour $n = 1000$.

```

1 program eratosthene
2   implicit none
3   integer, parameter      :: n = 1000
4   integer, dimension(n) :: tab
5   integer                 :: imax, i, j, nb_premiers
6
7   ! Initialisation
8   do i = 2, n
9     tab(i) = i
10  end do
11  tab(1) = 0 ! 1 n'est pas premier
12
13  ! Crible
14  imax = int(sqrt(real(n)))
15  do i = 2, imax
16    if (tab(i) /= 0) then
17      j = i + i
18      do while (j <= n)
19        tab(j) = 0
20        j = j + i
21      end do
22    end if
23  end do
24
25  ! Affichage et comptage
26  nb_premiers = 0
27  do i = 2, n
28    if (tab(i) /= 0) then
29      nb_premiers = nb_premiers + 1
30      write(*, '(I5)', advance='no') tab(i)

```

```

31     end if
32   end do
33   print *
34   write(*, '(A,I4,A,I4)') "Premiers 2 a ", n, " : ", nb_premiers
35 end program eratosthene

```

Il y a 168 nombres premiers entre 2 et 1000.

La complexité de l'algorithme est $O(n \log \log n)$.

Exercice 5 - Tri à bulles

★★

Exercice

Écrire un sous-programme Fortran `tri_bulles` qui trie un tableau de réels dans l'ordre croissant. L'algorithme compare des éléments consécutifs et les échange si nécessaire, jusqu'à ce qu'aucun échange ne soit effectué en un passage. Tester avec le tableau [0.76, 0.38, 0.42, 0.91, 0.25, 0.13, 0.52, 0.69, 0.76, 0.98].

```

1  subroutine tri_bulles(tab, n)
2    implicit none
3    integer, intent(in)      :: n
4    real, intent(inout)      :: tab(n)
5    real                     :: temp
6    logical                  :: tri_termine
7    integer                  :: i
8
9    do
10     tri_termine = .true.
11     do i = 1, n-1
12       if (tab(i) > tab(i+1)) then
13         temp      = tab(i)
14         tab(i)     = tab(i+1)
15         tab(i+1)  = temp
16         tri_termine = .false.
17       end if
18     end do
19     if (tri_termine) exit
20   end do
21 end subroutine tri_bulles
22
23 program test_tri
24   implicit none

```

```

25  integer, parameter :: n = 10
26  real :: tab(n) = [0.76, 0.38, 0.42, 0.91, 0.25, &
27                    0.13, 0.52, 0.69, 0.76, 0.98]
28  call tri_bulles(tab, n)
29  print *, "Tableau trie :", tab
30 end program test_tri

```

Résultat attendu : [0.13, 0.25, 0.38, 0.42, 0.52, 0.69, 0.76, 0.76, 0.91, 0.98].

Complexité en cas défavorable : $O(n^2)$.

Exercice 6 - Intégration numérique

★★★

Exercice

Écrire un programme Fortran qui calcule numériquement $I = \int_0^\pi \sin(x) dx$ en utilisant la méthode des trapèzes, pour un nombre de subdivisions N donné. Comparer avec la valeur exacte $I = 2$.

La règle des trapèzes : $I \approx \frac{h}{2} [f(a) + 2 \sum_{k=1}^{N-1} f(x_k) + f(b)]$ avec $h = (b - a)/N$ et $x_k = a + kh$.

```

1  program integration_trapeze
2  implicit none
3  real, parameter :: pi = 3.14159265358979
4  integer          :: N, k
5  real             :: a, b, h, x, integrale, valeur_exacte
6
7  a = 0.0;  b = pi
8  valeur_exacte = 2.0  ! valeur exacte de l'integrale de sin
9                        sur [0, pi]
10
11  print *, "    N          I_approx      Erreur"
12  print *, "-----"
13
14  do N = 10, 1000, 100
15    h = (b - a) / real(N)
16    ! Termes extremes
17    integrale = 0.5 * (sin(a) + sin(b))
18    ! Termes intermediaires
19    do k = 1, N-1
20      x = a + k * h
21      integrale = integrale + sin(x)

```

```

21     end do
22     integrale = integrale * h
23     write(*,'(I5, 2F14.8)') N, integrale, abs(integrale -
        valeur_exacte)
24     end do
25 end program integration_trapeze

```

Pour $N = 10$: erreur ≈ 0.0082 . Pour $N = 1000$: erreur $\approx 8 \times 10^{-7}$. La méthode des trapèzes est d'ordre 2 : l'erreur est en $O(h^2) = O(1/N^2)$.

2. Exercices avancés de programmation Fortran

Exercice 7 - Résolution d'un système linéaire 2x2

★

Exercice

Écrire un programme Fortran qui résout le système de 2 équations à 2 inconnues par la règle de Cramer :

$$\begin{cases} u_1x + v_1y = w_1 \\ u_2x + v_2y = w_2 \end{cases}$$

Le programme doit gérer le cas où le déterminant est nul.

```

1 program systeme_2x2
2   implicit none
3   real :: u1, v1, w1, u2, v2, w2
4   real :: delta, x, y
5
6   u1=2.0; v1=5.0; w1=7.0
7   u2=4.0; v2=11.0; w2=6.0
8
9   delta = u1*v2 - u2*v1
10  if (abs(delta) < 1.0e-6) then
11    print *, "Determinant nul: pas de solution unique."
12    stop
13  end if
14
15  x = (w1*v2 - w2*v1) / delta
16  y = (u1*w2 - u2*w1) / delta
17  print *, "Solution: x =", x, ", y =", y
18 end program systeme_2x2

```

Pour $u_1 = 2$, $v_1 = 5$, $w_1 = 7$, $u_2 = 4$, $v_2 = 11$, $w_2 = 6$: $\Delta = 22 - 20 = 2$,
 $x = (77 - 30)/2 = 23,5$, $y = (12 - 28)/2 = -8$.

Exercice 8 - Racines d'un trinôme

★

Exercice

Écrire un programme Fortran pour calculer les racines d'un trinôme du second degré $ax^2 + bx + c$. Le programme doit vérifier que $a \neq 0$ et gérer les trois cas selon le signe du discriminant.

```

1 program trinome
2   implicit none
3   real, parameter :: eps = 1.0e-6
4   real :: a, b, c, delta, x1, x2
5
6   a = 3.0; b = 7.0; c = -11.0
7
8   if (abs(a) < eps) stop "a doit etre non nul."
9
10  delta = b*b - 4.0*a*c
11
12  if (delta < -eps) then
13    print *, "Pas de racine reelle."
14  else if (abs(delta) < eps) then
15    x1 = -b / (2.0*a)
16    print *, "Racine double: x =", x1
17  else
18    x1 = (-b - sqrt(delta)) / (2.0*a)
19    x2 = (-b + sqrt(delta)) / (2.0*a)
20    print *, "Deux racines: x1 =", x1, ", x2 =", x2
21  end if
22 end program trinome

```

Pour $a = 3, b = 7, c = -11$: $\Delta = 49 + 132 = 181 > 0$. $x_1 = (-7 - \sqrt{181})/6 \approx -3,42$, $x_2 \approx 1,08$.

Exercice 9 - Suite de Fibonacci et nombre d'or

★★

Exercice

Écrire un programme Fortran qui calcule le nombre d'or à partir de la suite de Fibonacci définie par $u_0 = 1, u_1 = 1, u_{n+1} = u_n + u_{n-1}$. Le rapport u_{n+1}/u_n converge vers le nombre d'or $\varphi = (1 + \sqrt{5})/2$. Arrêter quand la variation relative est inférieure à 10^{-5} .

```

1 program nombre_dor
2   implicit none
3   real, parameter :: eps = 1.0e-5
4   real :: u_prec, u_cour, somme
5   real :: rapport_prec, rapport_cour
6   real :: phi_exact
7   integer :: n

```

```

8
9  phi_exact = (1.0 + sqrt(5.0)) / 2.0
10 u_prec = 1.0; u_cour = 1.0
11 rapport_prec = 1.0
12 n = 1
13
14 do
15     somme = u_cour + u_prec
16     u_prec = u_cour
17     u_cour = somme
18     rapport_cour = u_cour / u_prec
19     n = n + 1
20     if (abs(rapport_cour - rapport_prec) < eps) exit
21     rapport_prec = rapport_cour
22 end do
23
24 print *, "Iterations :", n
25 print *, "Approx      :", rapport_cour
26 print *, "Exact       :", phi_exact
27 print *, "Err rel     :", abs(rapport_cour - phi_exact)/
    phi_exact
28 end program nombre_dor

```

La convergence est atteinte en environ 25 itérations.

Le nombre d'or est $\varphi \approx 1,61803398\dots$

Exercice 10 - Crible d'Ératosthène

★★

Exercice

Écrire un programme Fortran qui détermine tous les nombres premiers dans l'intervalle $[2, N]$ à l'aide du crible d'Ératosthène. On rappelle l'algorithme : former un tableau de 2 à N , puis pour chaque entier non rayé, rayer tous ses multiples. La recherche peut s'arrêter à $\sqrt{N}$.

```

1 program eratosthene
2     implicit none
3     integer, parameter :: N = 200
4     integer :: tab(N), imax, i, j, count
5
6     ! Initialisation
7     do i = 2, N

```

```

8      tab(i) = i
9  end do
10     tab(1) = 0    ! 1 n'est pas premier
11
12     ! Crible
13     imax = int(sqrt(real(N)))
14     do i = 2, imax
15         if (tab(i) /= 0) then
16             j = i + i
17             do while (j <= N)
18                 tab(j) = 0
19                 j = j + i
20             end do
21         end if
22     end do
23
24     ! Affichage
25     count = 0
26     print *, "Nombres premiers entre 2 et", N, ":"
27     do i = 2, N
28         if (tab(i) /= 0) then
29             write(*, '(I5)', advance='no') tab(i)
30             count = count + 1
31         end if
32     end do
33     print *
34     print *, "Nombre total de premiers:", count
35 end program eratosthene

```

Il y a 46 nombres premiers entre 2 et 200. Le dernier est 199 (premier).

Exercice 11 - Méthode de Simpson pour l'intégration

★★★

Exercice

Écrire un programme Fortran qui calcule numériquement $I = \int_0^1 e^{-x^2} dx$ en utilisant la méthode de Simpson :

$$I \approx \frac{h}{3} [f(a) + 4f(x_1) + 2f(x_2) + 4f(x_3) + \cdots + f(b)]$$

où $h = (b - a)/N$ et N est pair. Comparer avec la valeur de référence $I \approx 0,74682$.

```

1 program simpson
2   implicit none
3   real, parameter :: pi = 3.14159265358979
4   integer :: N, k
5   real      :: a, b, h, x, integrale, ref
6
7   a = 0.0; b = 1.0
8   ref = 0.74682413 ! valeur de reference
9
10  print *, "      N          I_Simpson      Erreur"
11  print *, "-----"
12
13  N = 10
14  do while (N <= 10000)
15    if (mod(N, 2) /= 0) N = N + 1 ! N doit etre pair
16    h = (b - a) / real(N)
17
18    ! Termes extremes
19    integrale = exp(-a**2) + exp(-b**2)
20
21    ! Termes impairs (coefficient 4)
22    do k = 1, N-1, 2
23      x = a + k * h
24      integrale = integrale + 4.0 * exp(-x**2)
25    end do
26
27    ! Termes pairs (coefficient 2)
28    do k = 2, N-2, 2
29      x = a + k * h
30      integrale = integrale + 2.0 * exp(-x**2)
31    end do
32
33    integrale = integrale * h / 3.0
34    write(*, '(I6, 2F14.8)') N, integrale, abs(integrale - ref)
35    N = N * 10
36  end do
37 end program simpson

```

La méthode de Simpson est d'ordre 4 : l'erreur est en $O(h^4)$. Pour $N = 10$, erreur $\approx 10^{-6}$; pour $N = 100$, erreur $\approx 10^{-10}$.

Exercice 12 - Produit de deux matrices

★★

Exercice

Écrire un programme Fortran qui effectue le produit de deux matrices A ($n \times m$) et B ($m \times p$). Définir les dimensions à l'aide de paramètres. Tester avec $n = 3$, $m = 3$, $p = 3$, $A = I_3$ (identité) et B quelconque.

```

1 program produit_matrices
2   implicit none
3   integer, parameter :: n = 3, m = 3, p = 3
4   real :: A(n,m), B(m,p), C(n,p)
5   integer :: i, j, k
6
7   ! Initialisation de A = identite
8   A = 0.0
9   do i = 1, min(n, m)
10      A(i,i) = 1.0
11   end do
12
13   ! Initialisation de B (quelconque)
14   data ((B(i,j), j=1,p), i=1,m) / &
15       1.0, 2.0, 3.0, &
16       4.0, 5.0, 6.0, &
17       7.0, 8.0, 9.0/
18
19   ! Produit matriciel
20   do i = 1, n
21       do j = 1, p
22           C(i,j) = 0.0
23           do k = 1, m
24               C(i,j) = C(i,j) + A(i,k) * B(k,j)
25           end do
26       end do
27   end do
28
29   ! Affichage
30   print *, "Matrice C = A * B :"
31   do i = 1, n
32       write(*, '(3F8.2)') (C(i,j), j=1,p)
33   end do
34 end program produit_matrices

```

Puisque $A = I_3$, le résultat est $C = B$. Vérification : $C_{ij} = \sum_k \delta_{ik} B_{kj} = B_{ij}$.

3. Exercices avancés : Fichiers, matrices, méthodes numériques

Exercice 13 - Écrire à l'écran

★★

Exercice

Écrire un programme Fortran qui demande à l'utilisateur le nom d'un fichier texte, ouvre ce fichier, puis l'affiche à l'écran ligne par ligne en faisant précéder chaque ligne de son numéro. Tester avec le fichier source du programme lui-même.

```

1 program ligne_fichier
2   implicit none
3   integer :: i, err
4   character(len=30) :: fich, ncar
5   character(len=250) :: ligne
6   character(len=60) :: chaine
7
8   print*, 'Donner le nom du fichier : '
9   read*, fich
10
11  open(unit=10, file=fich)
12  err = 0
13  i = 1
14
15  do while (err == 0)
16    read(10, fmt='(1a250)', iostat=err) ligne
17    if (err /= 0) exit
18    ! Nombre de caracteres utiles (sans blancs debut/fin)
19    write(ncar, '(I3)') len(trim(adjustl(ligne)))
20    ! Construction du format de sortie
21    chaine = '(1I3, 1a, 1a' // trim(adjustl(ncar)) // ')'
22    write(*, fmt=chaine) i, ' ', trim(adjustl(ligne))
23    i = i + 1
24  end do
25
26  close(10)
27 end program ligne_fichier

```

Le programme utilise `iostat=err` pour détecter la fin de fichier (code $\neq 0$) sans provoquer d'erreur. Le format de sortie est construit dynamiquement à partir de la

longueur effective de chaque ligne.

Exercice 14 - Produit tensoriel

Exercice

Écrire un programme Fortran utilisant une fonction interne `rptens` qui renvoie le produit tensoriel de deux vecteurs réels de même taille n donnés en entrée. La taille des vecteurs n'est pas un argument de la fonction. Tester avec $\vec{u} = (1, 2, 3)$ et $\vec{v} = (2, 1, 3)$.

Le produit tensoriel de deux vecteurs $\vec{u}$ et $\vec{v}$ est la matrice $W_{ij} = u_i v_j$.

```

1 program prog_rptens
2   implicit none
3   integer, parameter :: n = 3
4   integer :: i
5   real, dimension(n) :: u, v
6   real, dimension(n, n) :: w
7   character(len=30) :: cn, chaine
8
9   u = (/1., 2., 3./); v = (/2., 1., 3./)
10
11   ! --- appel fonction
12   w = rptens(u, v)
13
14   ! --- affichage
15   write(cn, *) n
16   chaine = '(' // trim(adjustl(cn)) // 'F10.5)'
17   do i = 1, n
18     write(*, fmt=chaine) w(i, :)
19   end do
20
21 contains
22
23   function rptens(u, v) result(w)
24     implicit none
25     real, dimension(:), intent(in) :: u, v
26     real, dimension(size(u), size(v)) :: w
27     integer :: i, j
28     do i = 1, size(u)
29       do j = 1, size(v)
30         w(i, j) = u(i) * v(j)
31       end do

```

```

32     end do
33     end function rptens
34
35 end program prog_rptens

```

Pour $\vec{u} = (1, 2, 3)$ et $\vec{v} = (2, 1, 3)$, le résultat est :

$$W = \vec{u} \otimes \vec{v} = \begin{pmatrix} 2 & 1 & 3 \\ 4 & 2 & 6 \\ 6 & 3 & 9 \end{pmatrix}.$$

Exercice 15 - Produit de matrices (avancé)

Exercice

Écrire un programme Fortran qui effectue le produit de deux matrices entières A ($m_a \times n_a$) et B ($m_b \times n_b$) via :

1. Une sous-routine interne `matrix_mult` dont les arguments d'entrée sont A , B et leurs dimensions, et dont l'argument de sortie est C .
2. Une sous-routine interne `matrix_mult_bis` qui fait la même chose sans passer les dimensions en argument (tableaux allocatables).

Tester avec $m_a = 2, n_a = 3, m_b = 3, n_b = 2$.

```

1 program prog_mult
2   implicit none
3   integer, parameter :: ma=2, na=3, mb=3, nb=2
4   integer :: i
5   integer, dimension(ma, na) :: A
6   integer, dimension(mb, nb) :: B
7   integer, dimension(ma, nb) :: C
8
9   A = reshape((/1, 6, 4, 5, 2, 8/), (/2, 3/))
10  B = reshape((/6, 4, 2, 8, 4, 9/), (/3, 2/))
11
12  ! --- appel subroutine avec dimensions en arguments
13  call matrix_mult(A, B, C, ma, na, mb, nb)
14  print*, 'calcul avec matrix_mult :'
15  do i = 1, ma
16    print*, C(i, :)
17  end do
18

```

```

19  ! --- appel subroutine sans dimensions en arguments
20  call matrix_mult_bis(A, B, C)
21  print*, 'calcul avec matrix_mult_bis :'
22  do i = 1, ma
23      print*, C(i, :)
24  end do
25
26  contains
27
28  subroutine matrix_mult(A, B, C, ma, na, mb, nb)
29      implicit none
30      integer, intent(in)  :: ma, na, mb, nb
31      integer, dimension(ma, na), intent(in)  :: A
32      integer, dimension(mb, nb), intent(in)  :: B
33      integer, dimension(ma, nb), intent(out) :: C
34      integer :: i, j, k, coeff
35      do i = 1, ma
36          do j = 1, nb
37              coeff = 0
38              do k = 1, na
39                  coeff = coeff + A(i, k) * B(k, j)
40              end do
41              C(i, j) = coeff
42          end do
43      end do
44  end subroutine matrix_mult
45
46  subroutine matrix_mult_bis(A, B, C)
47      implicit none
48      integer, dimension(:, :), intent(in)  :: A
49      integer, dimension(:, :), intent(in)  :: B
50      integer, dimension(:, :), intent(out) :: C
51      integer :: i, j, k, coeff
52      do i = 1, size(A, 1)
53          do j = 1, size(B, 2)
54              coeff = 0
55              do k = 1, size(A, 2)
56                  coeff = coeff + A(i, k) * B(k, j)
57              end do
58              C(i, j) = coeff
59          end do

```

```

60     end do
61     end subroutine matrix_mult_bis
62
63 end program prog_mult

```

Les deux sous-routines donnent le même résultat. La version **bis** est plus générale car elle utilise **size()** pour récupérer les dimensions.

Exercice 16 - Résolution par dichotomie et Newton

★★★

Exercice

Soit la fonction $f(x) = x^3 - 4x - 2$ définie sur $\mathbb{R}$.

1. Montrer que f admet trois racines réelles x_1, x_2, x_3 dans les intervalles $[-2, -1]$, $[-1, 0]$ et $[2, 3]$.
2. Rechercher par dichotomie la solution de $f(x) = 0$ dans l'intervalle $[2, 3]$. Effectuer 3 itérations en indiquant la précision à chaque étape.
3. Reprendre la deuxième question en utilisant la méthode de Newton avec $x_0 = 4$: $x_{i+1} = x_i - f(x_i)/f'(x_i)$.

1. Vérification des racines. On calcule les valeurs de f aux bornes :

- . Sur $[-2, -1]$: $f(-2) = -8 + 8 - 2 = -2 < 0$ et $f(-1) = -1 + 4 - 2 = 1 > 0$.
Signe opposé $\Rightarrow$ racine.
- . Sur $[-1, 0]$: $f(-1) = 1 > 0$ et $f(0) = -2 < 0$. Signe opposé $\Rightarrow$ racine.
- . Sur $[2, 3]$: $f(2) = 8 - 8 - 2 = -2 < 0$ et $f(3) = 27 - 12 - 2 = 13 > 0$. Signe opposé $\Rightarrow$ racine.

2. Méthode de dichotomie sur $[a_0, b_0] = [2, 3]$:

i	a_i	b_i	x_i	$f(x_i)$	$e_i = (b_i - a_i)/2$
0	2	3	2.5	3.625	0.5
1	2	2.5	2.25	0.3906	0.25
2	2	2.25	2.125	-0.9043	0.125
3	2.125	2.25	2.188	-0.2825	0.0625

Après 3 itérations, $x^* \approx 2.188$.

3. Méthode de Newton avec $f'(x) = 3x^2 - 4$ et $x_0 = 4$:

i	x_i	x_{i+1}	$ x_{i+1} - x_i $
0	4	2.95455	1.04545
1	2.95455	2.41493	0.53962
2	2.41493	2.23533	0.1796
3	2.23533	2.21459	0.02074

La solution est $x^* \approx 2.21459$. Newton converge plus rapidement que la dichotomie.

Exercice 17 - Mouvement d'un objet en chute libre

★★

Exercice

Un objet est lâché depuis une hauteur h_0 à partir du repos ($v_0 = 0$) sous l'influence de la gravité. On néglige la résistance de l'air. L'équation du mouvement est :

$$h(t) = h_0 - \frac{1}{2}gt^2$$

où h_0 est la hauteur initiale (en m), $g = 9,81 \text{ m s}^{-2}$ et $h(t)$ la hauteur à l'instant t .

On veut déterminer le temps t nécessaire pour que l'objet atteigne le sol ($h(t) = 0$).

1. Vérifier s'il existe une racine de $f(t) = h(t) = 0$ dans l'intervalle $[0, 10]$ secondes.
2. Combien d'itérations de dichotomie sont nécessaires pour une précision de 10^{-3} ?
3. Appliquer la méthode de dichotomie pour $h_0 = 100 \text{ m}$ et $g = 9,81 \text{ m.s}^{-2}$ avec une tolérance de 0,05.

1. On pose $f(t) = h_0 - \frac{1}{2}gt^2$. Pour $h_0 = 100 \text{ m}$:

$$f(0) = 100 > 0, \quad f(10) = 100 - \frac{1}{2} \times 9,81 \times 100 = -390,5 < 0.$$

f change de signe sur $[0, 10] \Rightarrow$ il existe bien une racine.

2. Le nombre d'itérations pour une précision $\varepsilon = 10^{-3}$:

$$n = \frac{\log(b-a) - \log(\varepsilon)}{\log 2} = \frac{\log(10) + 3}{\log 2} \approx \frac{4}{0,301} \approx 13,3.$$

Il faut donc environ **14 itérations**.

3. Application de la dichotomie ($h_0 = 100$, tolérance 0,05) :

i	a_i	x_i	b_i	$f(x_i)$	e_i
1	0.0000	5.0000	10.000	-22.625	5.0000
2	0.0000	2.5000	5.0000	69.3437	2.5000
3	2.5000	3.7500	5.0000	31.0234	1.2500
4	3.7500	4.3750	5.0000	6.1152	0.6250
5	4.3750	4.6875	5.0000	-7.7759	0.3125
6	4.3750	4.5313	4.6875	-0.7106	0.1562
7	4.3750	4.4531	4.5312	2.7323	0.0781
8	4.4531	4.4922	4.5312	1.0183	0.0391

Le temps de chute est $t \approx 4,4922$ s. La valeur exacte est $t = \sqrt{2h_0/g} = \sqrt{200/9,81} \approx 4,515$ s.

Exercice 18 - Résolution d'une équation non-linéaire

★★★

Exercice

On se propose de résoudre numériquement l'équation $f(x) = 0$ par la méthode de la bisection (dichotomie) et par la méthode de Newton-Raphson.

1. Écrire un programme Fortran qui résout l'équation par la méthode de la dichotomie.
2. Écrire un programme Fortran qui résout l'équation par la méthode de Newton.
3. Tester les deux programmes pour l'équation $x - \cos(x) = 0$ sur l'intervalle $[0,5 ; 1,5]$ avec une précision $\varepsilon = 10^{-4}$.
4. Comparer les résultats des deux méthodes.

1. Programme Fortran pour la dichotomie :

```

1 program dichotomie
2   implicit none
3   real :: a, b, eps, x, fa, fx
4   integer :: niter
5
6   print*, 'Donner les deux bornes a et b :'
7   read*, a, b
8   print*, 'Donner la precision eps :'
9   read*, eps
10
11   fa = fct(a)
12   niter = 0
13
```

```

14  do while (abs(b - a) > eps)
15      x = (a + b) / 2.0
16      fx = fct(x)
17      if (fa * fx < 0.0) then
18          b = x
19      else
20          a = x
21          fa = fx
22      end if
23      niter = niter + 1
24  end do
25
26  print*, 'La solution est x =', x
27  print*, 'Nombre d''iterations :', niter
28
29  contains
30
31  function fct(x)
32      implicit none
33      real :: fct, x
34      fct = x - cos(x)
35  end function fct
36
37  end program dichotomie

```

2. Programme Fortran pour Newton-Raphson :

```

1  program Newton
2      implicit none
3      real :: x, x0, eps, fx, fpx
4      integer :: niter
5
6      print*, 'Donner la valeur initiale x0 :'
7      read*, x0
8      print*, 'Donner la precision eps :'
9      read*, eps
10
11     x = x0
12     niter = 0
13
14     do while (abs(fct(x)) > eps)
15         fx = fct(x)

```

```

16     fpx = fctp(x)
17     if (fpx == 0.0) then
18         print*, 'Derivee nulle, arret.'
19         stop
20     end if
21     x = x - fx / fpx
22     niter = niter + 1
23 end do

24
25 print*, 'La solution est x =', x
26 print*, 'Nombre d''iterations :', niter
27
28 contains
29
30     function fct(x)
31         implicit none
32         real :: fct, x
33         fct = x - cos(x)
34     end function fct
35
36     function fctp(x)
37         implicit none
38         real :: fctp, x
39         fctp = 1.0 + sin(x)
40     end function fctp
41
42 end program Newton

```

3 & 4. Comparaison. Sur $[0,5; 1,5]$ avec $\varepsilon = 10^{-4}$:

La dichotomie requiert environ $n = \lceil \log_2((1,5 - 0,5)/10^{-4}) \rceil = 14$ itérations. Newton, avec $x_0 = 1$, converge en **3–4 itérations** vers la solution $x^* \approx 0,7391$, qui vérifie $\cos(0,7391) \approx 0,7391$.

Newton est nettement plus rapide (convergence quadratique) mais nécessite de connaître la dérivée $f'(x) = 1 + \sin(x)$.